



DFSMS Advanced Topics: Tape Policy management

Session 2450

Mike Wood

Mikew_wood@uk.ibm.com

Abstract: DFSMSrmm tape policy management is relevant to archiving and compliance as well as business continuity. In this session, the speaker will review function for managing the lifecycle of data on tape and the tapes themselves. He will cover the most recent tape policy management features and how to exploit them.



NOTICES AND DISCLAIMERS

Copyright © 2009 by International Business Machines Corporation.

No part of this document may be reproduced or transmitted in any form without written permission from IBM Corporation.

Product information and data has been reviewed for accuracy as of the date of initial publication. Product information and data is subject to change without notice. This document could include technical inaccuracies or typographical errors. IBM may make improvements and/or changes in the product(s) and/or programs(s) described herein at any time without notice.

References in this document to IBM products, programs, or services does not imply that IBM intends to make such products, programs or services available in all countries in which IBM operates or does business. Consult your local IBM representative or IBM Business Partner for information about the product and services available in your area.

Any reference to an IBM Program Product in this document is not intended to state or imply that only that program product may be used. Any functionally equivalent program, that does not infringe IBM's intellectual property rights, may be used instead. It is the user's responsibility to evaluate and verify the operation of any non-IBM product, program or service.

THE INFORMATION PROVIDED IN THIS DOCUMENT IS DISTRIBUTED "AS IS" WITHOUT ANY WARRANTY, EITHER EXPRESS OR IMPLIED. IBM EXPRESSLY DISCLAIMS ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR INFRINGEMENT. IBM shall have no responsibility to update this information. IBM products are warranted according to the terms and conditions of the agreements (e.g., IBM Customer Agreement, Statement of Limited Warranty, International Program License Agreement, etc.) under which they are provided. IBM is not responsible for the performance or interoperability of any non-IBM products discussed herein.

The performance data contained herein was obtained in a controlled, isolated environment. Actual results that may be obtained in other operating environments may vary significantly. While IBM has reviewed each item for accuracy in a specific situation, there is no guarantee that the same or similar results will be obtained elsewhere.

The responsibility for use of this information or the implementation of any of these techniques is a customer responsibility and depends on the customer's or user's ability to evaluate and integrate them into their operating environment. Customers or users attempting to adapt these techniques to their own environments do so at their own risk. IN NO EVENT SHALL IBM BE LIABLE FOR ANY DAMAGE ARISING FROM THE USE OF THIS INFORMATION, INCLUDING BUT NOT LIMITED TO, LOSS OF DATA, BUSINESS INTERRUPTION, LOSS OF PROFIT OR LOSS OF OPPORTUNITY.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not necessarily tested those products in connection with this publication and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

The provision of the information contained herein is not intended to, and does not, grant any right or license under any IBM patents or copyrights. Inquiries regarding patent or copyright licenses should be made, in writing, to:

IBM Director of Licensing
IBM Corporation
North Castle Drive
Armonk, NY 10504-1785



Trademarks

The following terms are trademarks of International Business Machines Corporation in the United States, other countries, or both:

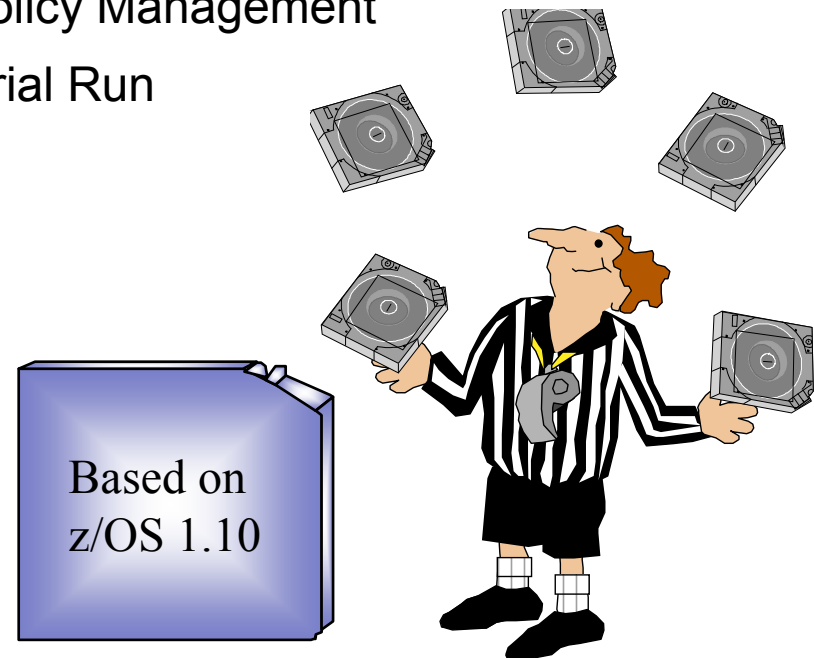
- DFSMS
- DFSMSdfp
- DFSMSdss
- DFSMSHsm
- DFSMSrmm
- DFSORT
- IBM
- RACF
- TotalStorage
- z/OS

Other company, product or service names may be trademarks or service marks of others.

Overview

- Retaining Data and Managing Life Cycles
- Policy Implementation
 - Inventory management
- Vital Record Specifications (VRS)
- Matching Policies to
 - Data Sets
 - Volumes
 - SMS Management Class
 - Management Value
- Retention Management
 - Retention Types
 - Multiple Retention Types
 - Retention Subchains

- Location Management / Vaulting
 - Stacked volume support
- Parmlib options
 - Volume Set Management
- Policy Management Trial Run



Retaining Data

- **Common Reasons for Retention on Tape**
 - Archiving
 - Compliance
 - Business Recovery and Continuity (BRC)
 - Versioning
- **Retention**
 - Enforceable or Flexible?
- **Lifecycle Management**
 - Data Sets
 - From Creation through Expiration
 - Volumes
 - From Manufacture through Deployment and Retirement/Replacement

Defining Retention Policies

- **Select A Strategy**
 - Enforce policies using VRS
 - User Specifies Policy in JCL
 - Support a Combination based on need
- **Enforce Policies using**
 - RELEASEOPTIONS
 - MASTEROVERWRITE
 - RACF FACILITY CLASS
- **Support User Policy Selection Using**
 - JCL RETPD/EXPDT/ACCODE
 - For example, EXPDT=99000
 - Parmlib MAXRETPD Can Control RETPD Values
 - SMSACS/EDGUX100 may be Required for EXPDT/ACCODE special values
- **Decide The Service Levels Required**
 - Each Service Level is implemented using a VRS chain
- **Set Expectations**
 - How Much new Data is Policy Retained
 - How Much Policy Retained Data Expires



Setting Service Levels

- **Reason For Retention**
 - Archiving, Compliance, BRC, ...
- **Enforceable or Flexible**
 - Read only
 - Allow/Prevent Ignore/EXPDT=98000
- **Type of Retention**
 - Time, Copies, Catalog, ...
- **Where to be Stored**
 - Local, vault, VTS, ...
- **Identifying Data**
 - Application, Data Set Name Mask, ...
- **Define the VRS Chain**
- **Determine Parmlib Options**



Set Retention Expectations

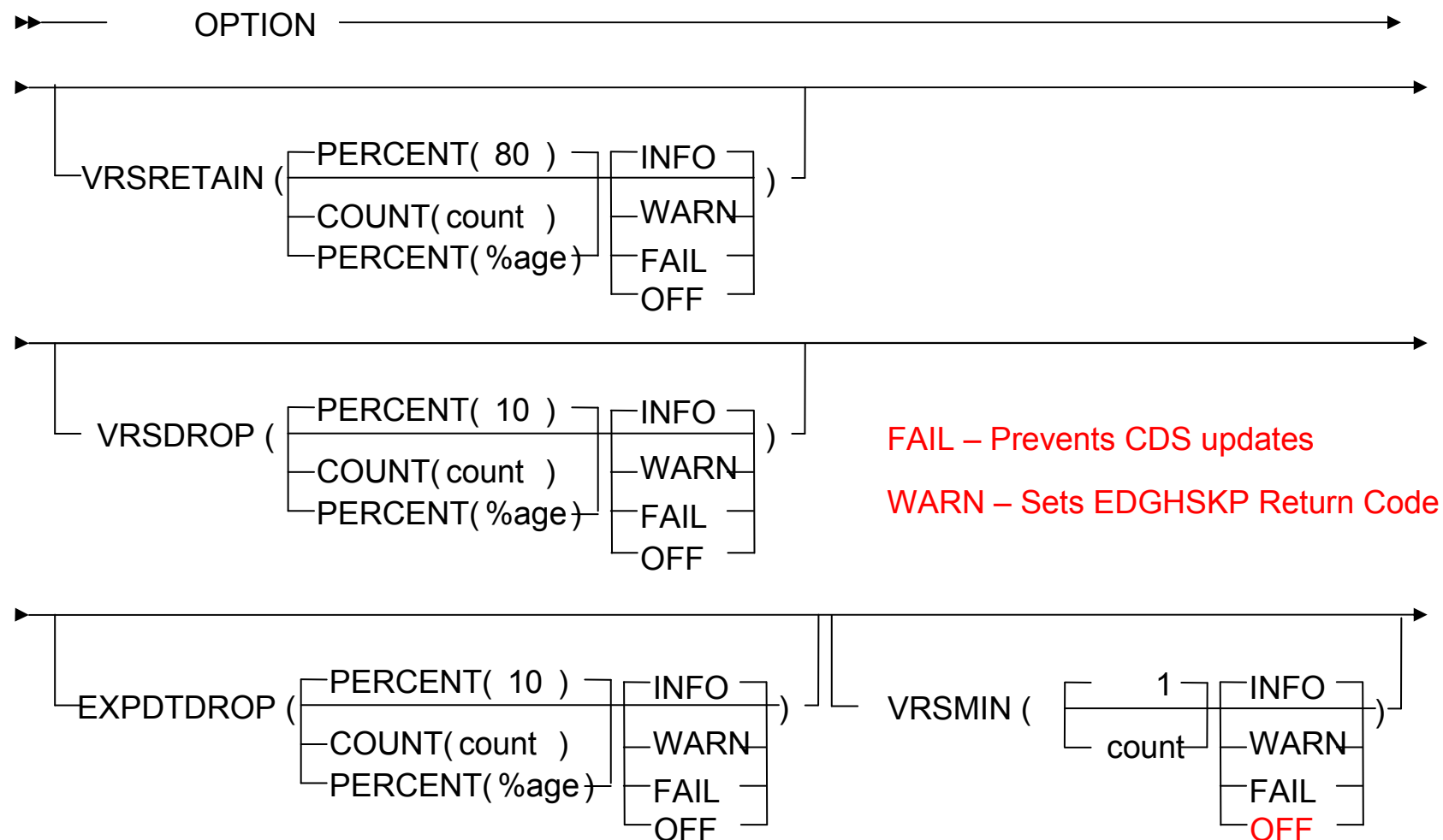
- **Avoiding Loss of Data**

- Pre-empt system errors, user errors
- Define expected retention as an overall control
- Exploit automated way, to detect when too many volumes are released or expired

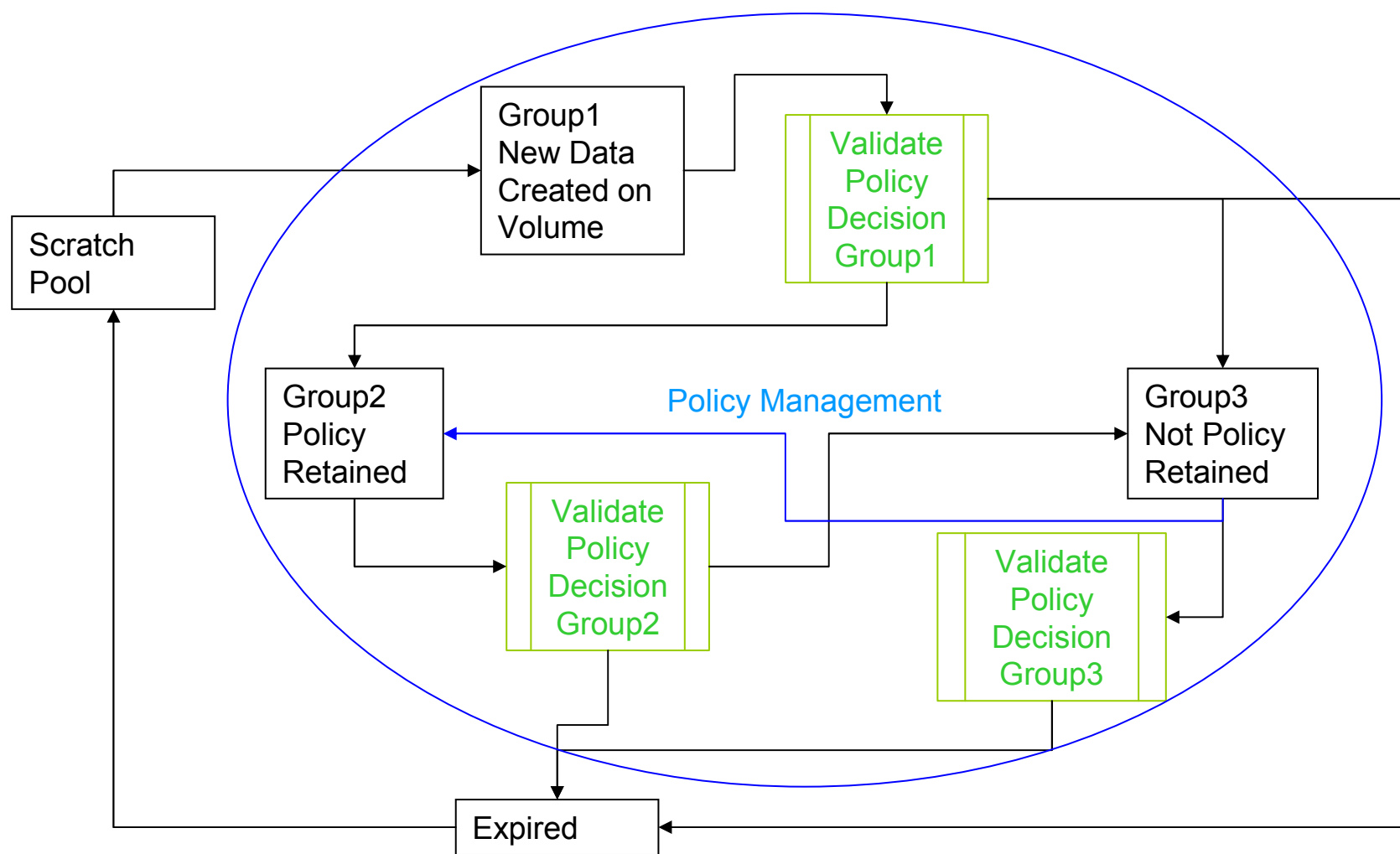
- **New parmlib options are added with z/OS V1.10**

- Use VRSDROP to control volumes dropped from VRS retention
- Use VRSRETAIN to control newly assigned volumes that must be retained by VRS
- Use EXPDTDROP to control volumes dropped from EXPDT retention

Set Retention Expectations . . .



Life Cycle for Removable Media Data



Set Retention Expectations . . .

■ MESSAGE file

```

.....
EDG2420I PHYSICAL VOLUMES READ           =          150   75%  ←
EDG2420I LOGICAL   VOLUMES READ           =           30   15%  ←
EDG2420I STACKED   VOLUMES READ           =           20   10%  ←
EDG2420I TOTAL     VOLUMES READ           =          200  100%  ←
.....
EDG2242I INITIAL NUMBER OF VRS RETAINED VOLUMES =          20   40%  ←
EDG2244I NUMBER OF VRS RETAINED VOLUMES TO BE DROPPED =           2   10%  ←
EDG2243I INITIAL NUMBER OF NEWLY ASSIGNED VOLUMES =          15   30%  ←
EDG2245I NUMBER OF NEWLY ASSIGNED VOLUMES TO BE RETAINED =           5   33%  ←
EDG2427I INITIAL NUMBER OF EXPDT RETAINED VOLUMES =          10  100%  ←
EDG2428I NUMBER OF EXPDT RETAINED VOLUMES TO BE DROPPED =           2   20%  ←
.....
EDG2421I PHYSICAL VOLUMES UPDATED         =           5   33%  ←
EDG2421I LOGICAL   VOLUMES UPDATED         =           1   33%  ←
EDG2421I STACKED   VOLUMES UPDATED         =           1   50%  ←
EDG2421I TOTAL     VOLUMES UPDATED         =           7   35%  ←

```

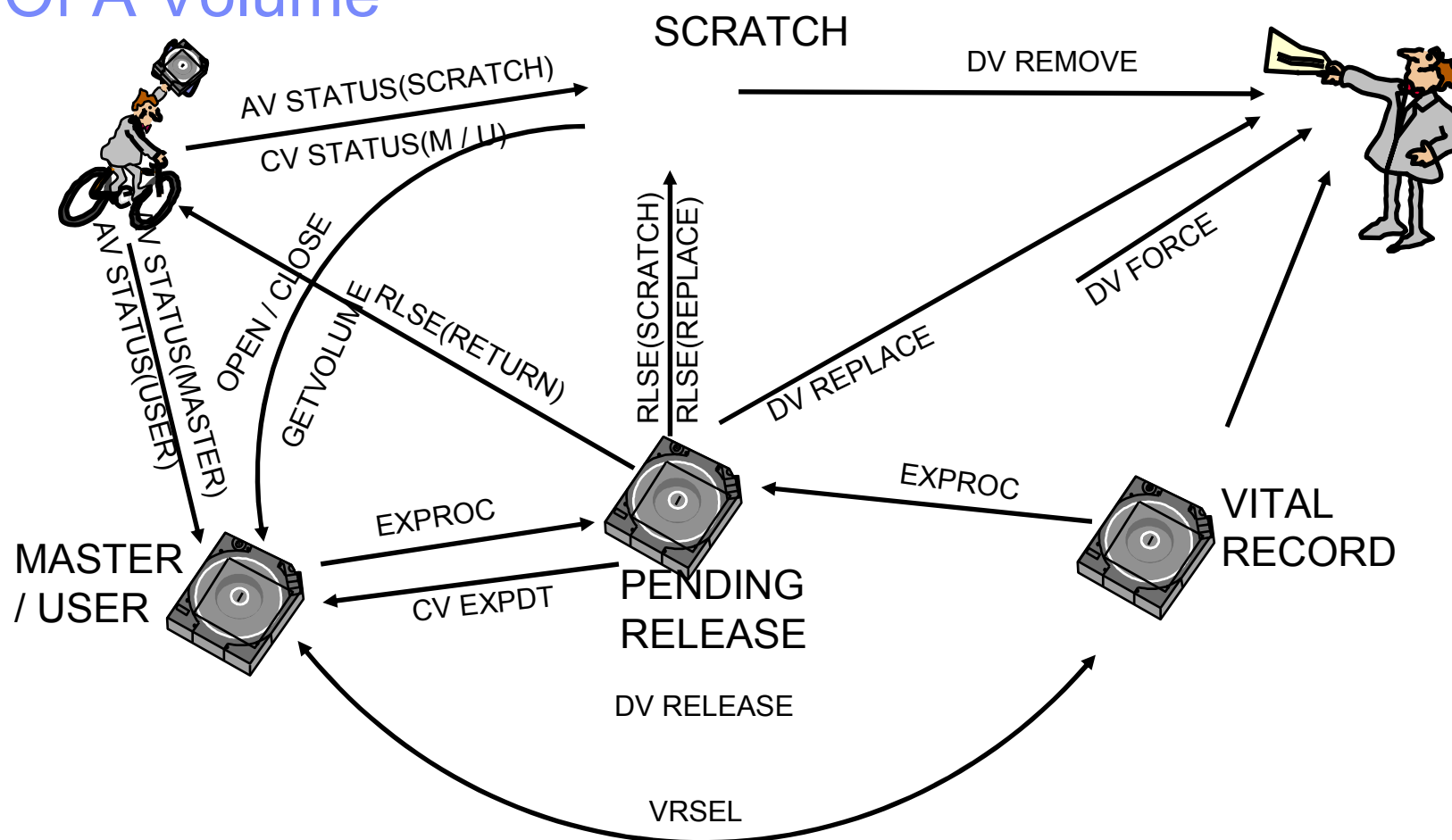


Archiving and Compliance

■ Special Needs for Enforcement

- Long term retention, prevention of Expiration
 - overridden temporarily for audit, investigations and legal reasons
 - Volume Level Volume VRS
 - Data Set Level MV/MC VRS
 - Pool Level EXPROC by Volume subset
- Prevent manual Release of Volumes
 - Limit Authorization via FACILITY CLASS
- Low Reference, Read only
 - OPTION/VLPOOL MASTEROVERWRITE(ADD) or WORM
- No Use Outside DFSMSrmm Control
 - Prevent 'Ignore' of Volume
 - Do not define OPENRULE with action(IGNORE)
 - EXPDT=98000
 - > Do not use EDGUX100
 - > Limit Authorization via FACILITY CLASS

Life Of A Volume





Media Life Cycle - Replacement

Prior to V1R10:

- Hard coded limit of '1' for permanent tape i/o errors Drives REPLACE release action
- USERMOD is available to set the limit to another value or disable support
- Non-Scratch Volumes are checked during Inventory Management and the REPLACE release action is set

V1R10 :

- New **Volume Replacement Policies** are added to parmlib to enable media end-of-life to be managed based on
 - permanent and temporary read and write errors detected by the tape hardware
 - volume usage
 - age
- RMM exploits tape drive **SARS MIM alerts** to identify active volumes which need replacement
- Non-Logical Volumes are checked during Inventory Management and the REPLACE release action is set
- Volumes with REPLACE release action are READ ONLY

Benefit:

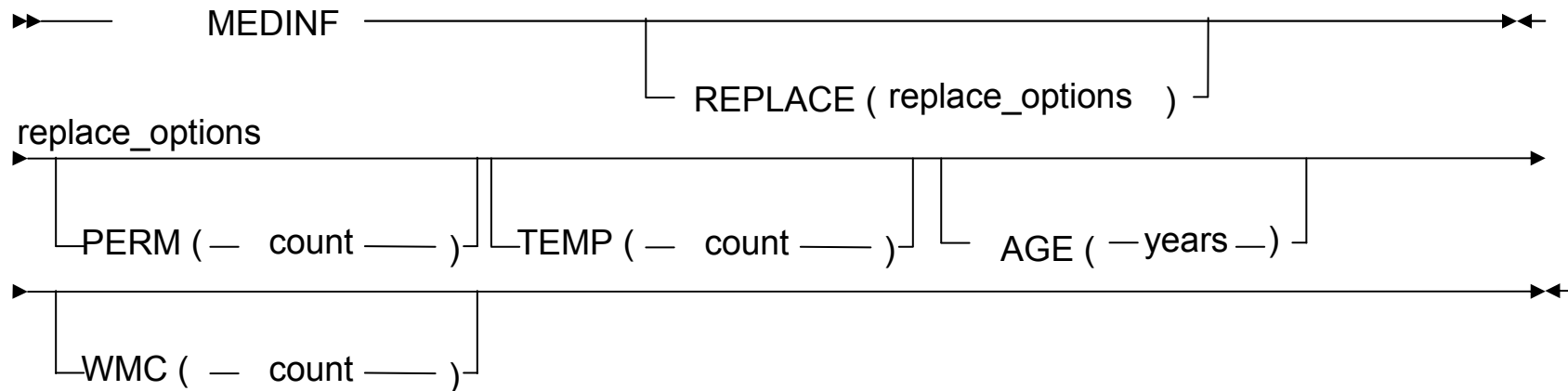
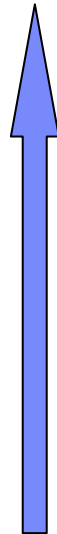
- Volume Replacement can be **easily customized**
- **Automated end-of-life management** of **active** and **inactive**, scratch and private volumes
- IBM Tivoli Tape Optimizer on z/OS V2.1 supports copying of non-expired volumes with the REPLACE release action

Media Life Cycle – Replacement . . .

■ Policies Can be Defined At:

- Default
 - REPLACE(PERM(1))
- Global Level
 - For example MEDINF NAME(IBM) REPLACE(PERM(0)) disables management
- Media Type
 - For example MEDINF NAME(ANO) MEDIATYPE(5,SQUARE) REPLACE(AGE(10))
- Specific Media Type and Recording Format
 - For example MEDINF NAME(ANO) MEDIATYPE(5,SQUARE) RECORDINGFORMAT(1,STD) REPLACE(WMC(999))

Go from
most specific
to less
specific



Media Life Cycle – Replacement . . .

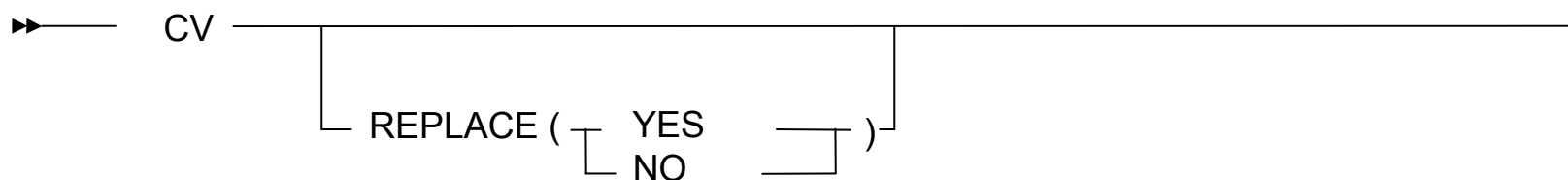
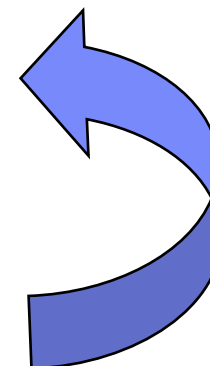
LC output of examples: Different types of Volume Replacement Policies

Media Information :									
						Replace Policy			
Name	Media type	Recording format	Capacity (MB)	Perm	Temp	WMC	Age		
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----
1. → IBM	3 HPCT	0 *	0	1	5	5000	10		
2. → NONIBM	1 STANDARD	3 36BTRACK	400	1	4	10000	15		
3. → NONIBM	0 *	0 *	0	2	8	0	20		

- 1. Media type level policy
- 2. Specific policy
- 3. Global policy

Media Life Cycle – Replacement . . .

- **Inventory Management for REPLACE**
 - Physical, Stacked, WORM; Master, User, or Scratch
 - not pending release, release action SCRATCH or WORM volumes
 - Release Action set to REPLACE
- **SARS MIM for Active Volumes**
 - Release Action set to REPLACE
- **Follow up the REPLACE Action/Release Action**
 - Volume Expired and Pending Release
 - Replace the volume and confirm replacement
 - Cancel the REPLACE action
 - Delete the Volume
 - Volume Contains active Data
 - Copy Data to a Newer Volume
 - Release the Volume



Media Life Cycle – Replacement . . .

■ Exploitation of SARS MIM alerts

- **SARS** : Statistical Analysis & Recording System used by IBM tape drives for tracking media statistics to determine /predict media failure
- SARS Media Information Messages (**MIM**) alerts occur at the time a volume is used
 - An eventual action message issued to the console
 - WTO message **ID IEA486E** is intercepted by DFSMSrmm
 - Error reference code (**refc**) **60-62 or 64** is used to set volume release action from SCRATCH to **REPLACE**

IEA486E *cuu*, TVOL, {ACUTE | SERIOUS | MODERATE | SERVICE} ALERT,
VOLUME=*valid*, S=*s*, MC=*mc*, E=*e*, REF=*refc-med-fi*, REPEATED

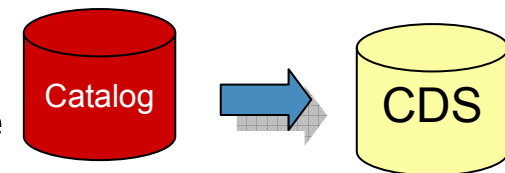
Policy Implementation

■ Inventory Management

- Vital Record Selection (PARM=VRSEL)
 - Dynamically applies Policies to Data Sets and Volumes and manages Retention and Movement



- Catalog Synchronization (PARM=CATSYNCH)
 - Sets Catalog Status for all Data Sets recorded in the CDS
 - VERIFY - reports on Differences
 - Runs once to Implement
 - Keeps in synch through catalog interface

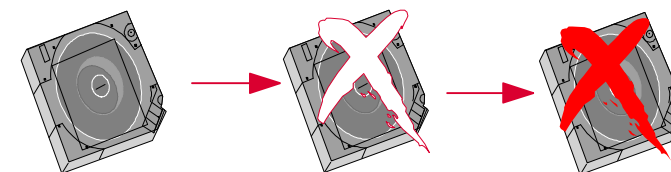


Policy Implementation . . .

■ Inventory Management . . .

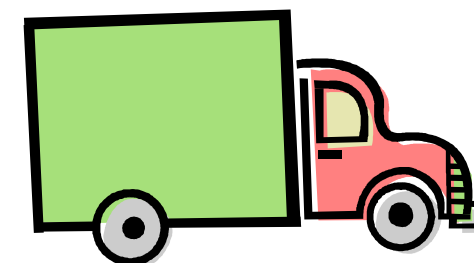
— Expiration Processing (PARM=EXPROC)

- Confirms Global Actions
- Implements Volume REPLACEment Policy
- Releases Volumes (Set Pending Release)
- Returns Volumes to Scratch
- Updates related information
 - Catalog
 - RACF
 - TCDB (SMS Tape Library)
- Optionally select volume subsets by volser and location



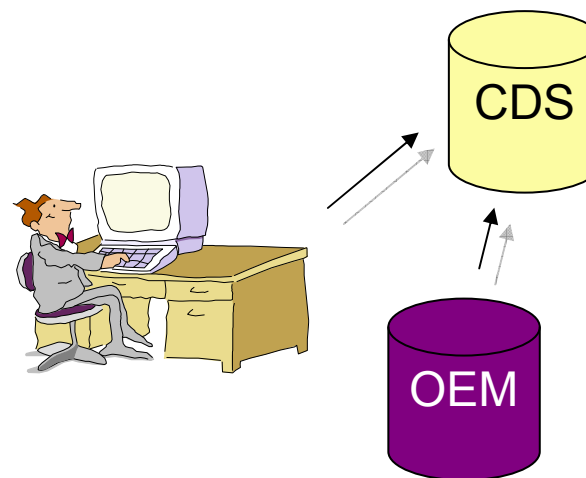
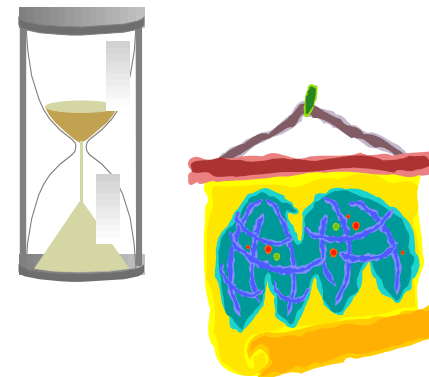
— Storage Location Management (PARM=DSTORE)

- Initiates/Confirms Moves
 - Optionally by Location
- Assigns and releases Bins
 - Optimized Bin Assignment to minimize slots required
 - Parmlib - REUSEBIN option



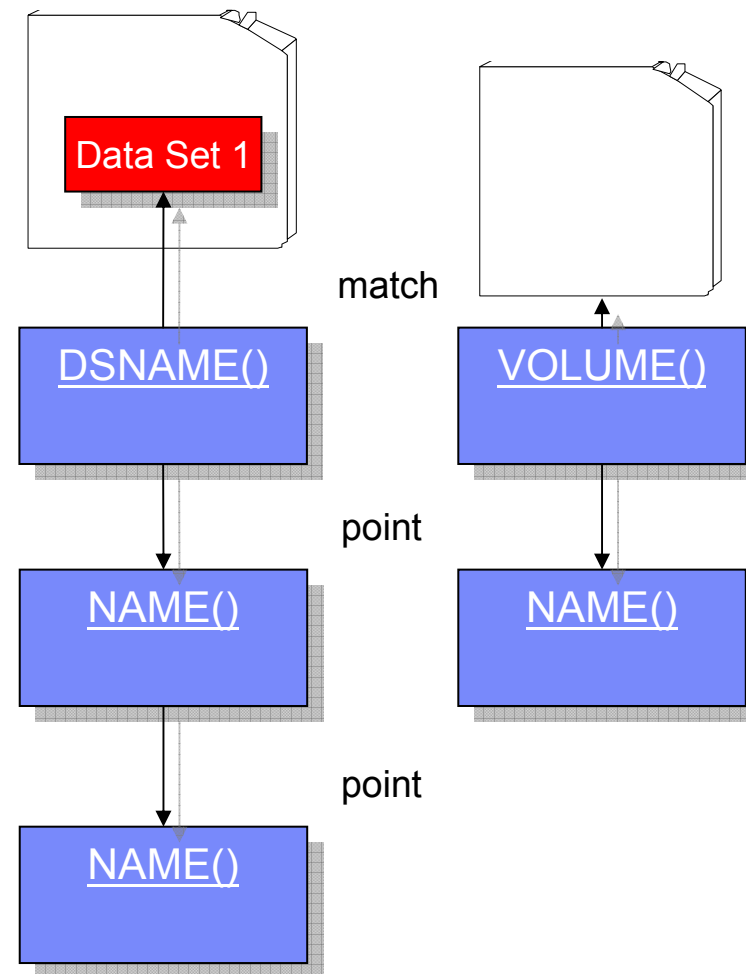
Vital Record Specification (VRS)

- **VRS's contain Policies to ...**
 - Control retention of data sets and volumes
 - Determine volume's required location
 - Set release options
- **VRS's are kept in the Control Data Set**
- **VRS's are created / maintained with**
 - Commands
 - ISPF Panels
 - Conversion tools



VRS Types

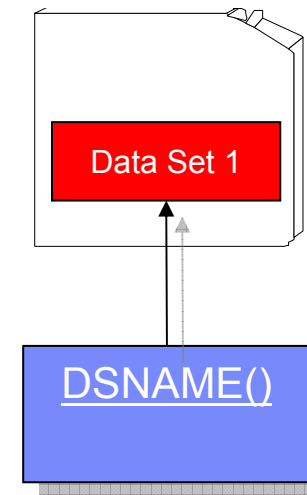
- **Data Set VRS**
 - Assigned to Data Sets
- **Volume VRS**
 - Assigned to Volumes
- **Name VRS**
 - Pointed by other VRS's
 - Retention Name VRS
 - Location Name VRS



Matching to a Data set VRS

■ Data Set Name

- Fully qualified data set name
- Data set name mask
- Single qualifier data set name mask
 - Management Class
 - Management Value
 - Reserved names
 - “OPEN“, “DELETED“, “ABEND“



■ Jobname

- Specific job name
- Job name mask
 - Reserved names
 - “OPEN“, “DELETED“, “ABEND“

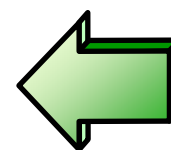
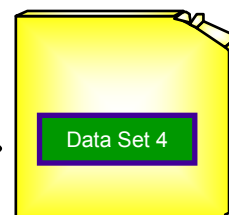
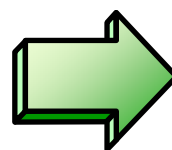
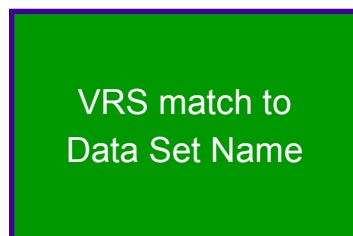
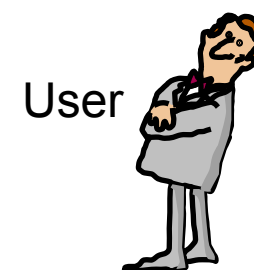
```
RMM ADDVRS DSNAME('PRITCHAR.BACKUP.DATA') DAYS COUNT(5)
RMM ADDVRS DSNAME('**') JOBNAME(LAURAN)
RMM ADDVRS DSNAME('**') JOBNAME(ABEND) DAYS COUNT(1)
```

■ GDG / NOGDG

Matching Order of Precedence

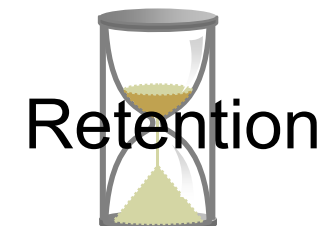
Primary VRS	Secondary VRS
OPEN VRS & Data Set is open	
DELETED VRS & DISP=DELETE	
ABEND VRS & Job abended	
Dsname VRS	Management Class VRS
Dsname VRS	Management Value VRS
Dsname VRS	
Management Class VRS	
Management Value VRS	
Default VRS – DSN('**')	

Option - Two Policies manage one data set



Primary VRS

Secondary VRS



Management Value

- **Management Value gets assigned to a Data Set by**
 - DFSMSrmm User Exit EDGUX100
 - Special Values in JCL parameter EXPDT=, ACCODE=
 - TSO RMM CHANGEDATASET command
 - Conversion to DFSMSrmm

Recommendation:

Use SMS Management Class to replace VRS Management Value

Management Class

- **Management Class gets assigned to a Data Set by**
 - SMS ACS (Automatic Class Selection)
 - JCL Parameter
 - TSO RMM CHANGEDATASET command
 - Conversion to DFSMSrmm

Recommendation:

Use SMS ACS FILTLISTs to replace RMM VRS Dsname Masks

Matching to a Volume VRS

- **By Specific Volume Serial Number**

- Retention is always DAYS since Creation of the first data set on the volume (assigned from scratch date)
- Retain the volume 10 days

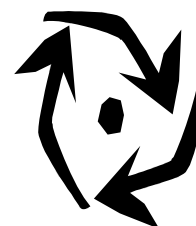


VOLUME(ABC001)

COUNT(10)

- **By Generic Volume Serial Number**

- Retention is always CYCLES
- Retain the „newest“ 5 volumes from those starting with a Volser of ABC



VOLUME(ABC*)

COUNT(5)

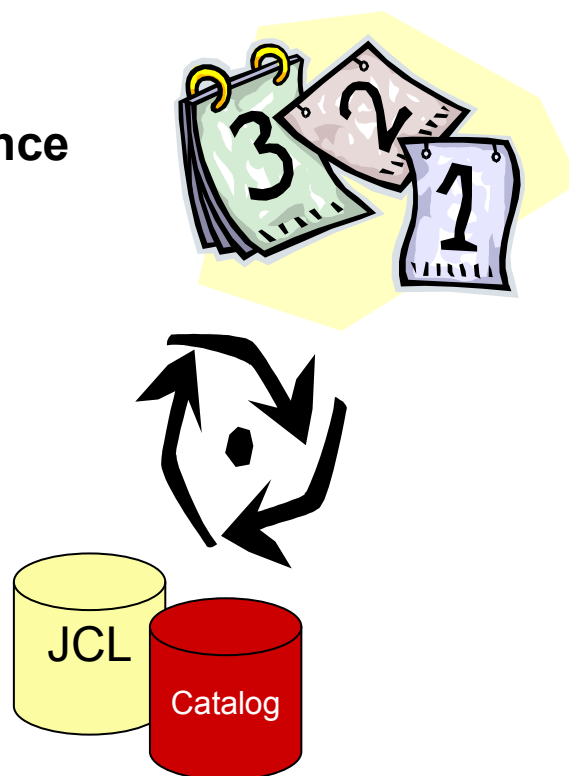
Retention Types

- **Specific Date**
- **Days since Creation**
- **Days since Last Reference**
- **Extra Days**

- **Cycles**
- **By Days Cycles**

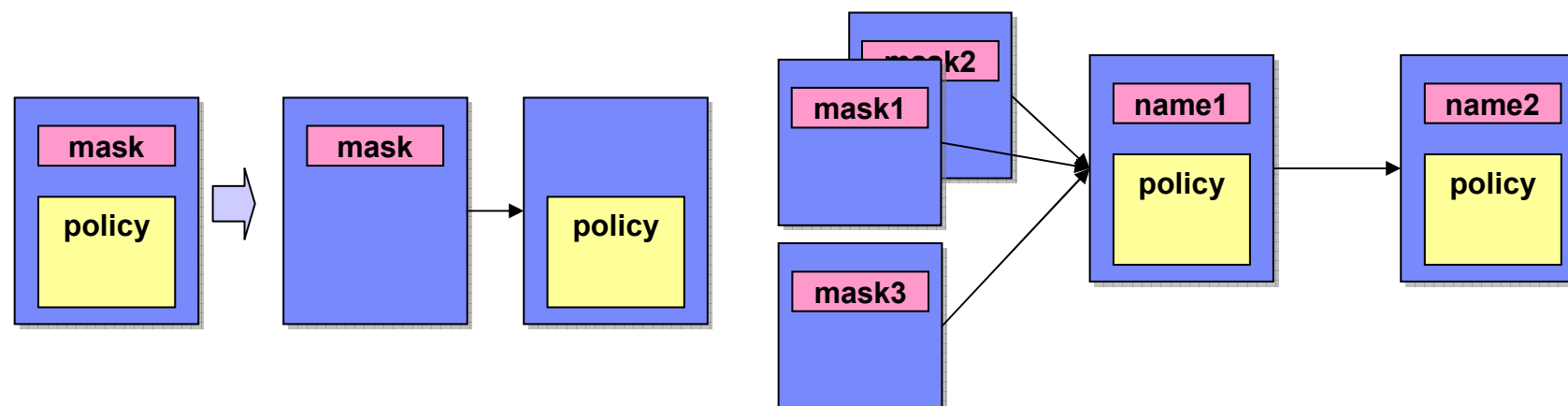
- **Until Expired**
- **While Cataloged**

- **Forever**
- **Don't retain**



Reusable DSNAMES Policies

- **Separation** of the data set name-job name **mask** from the **retention/movement details**
- **Avoiding duplication** in VRS definitions
- When adding a new data set to VRS retention, the process is **simple** because you:
 - Select an existing service level represented by a VRS chain that starts with a retention NAME VRS
 - Define the data set name VRS, specifying DSNNAME, JOBNAME, and GDG/NOGDG as required, with COUNT(0) and NEXTVRS(name)

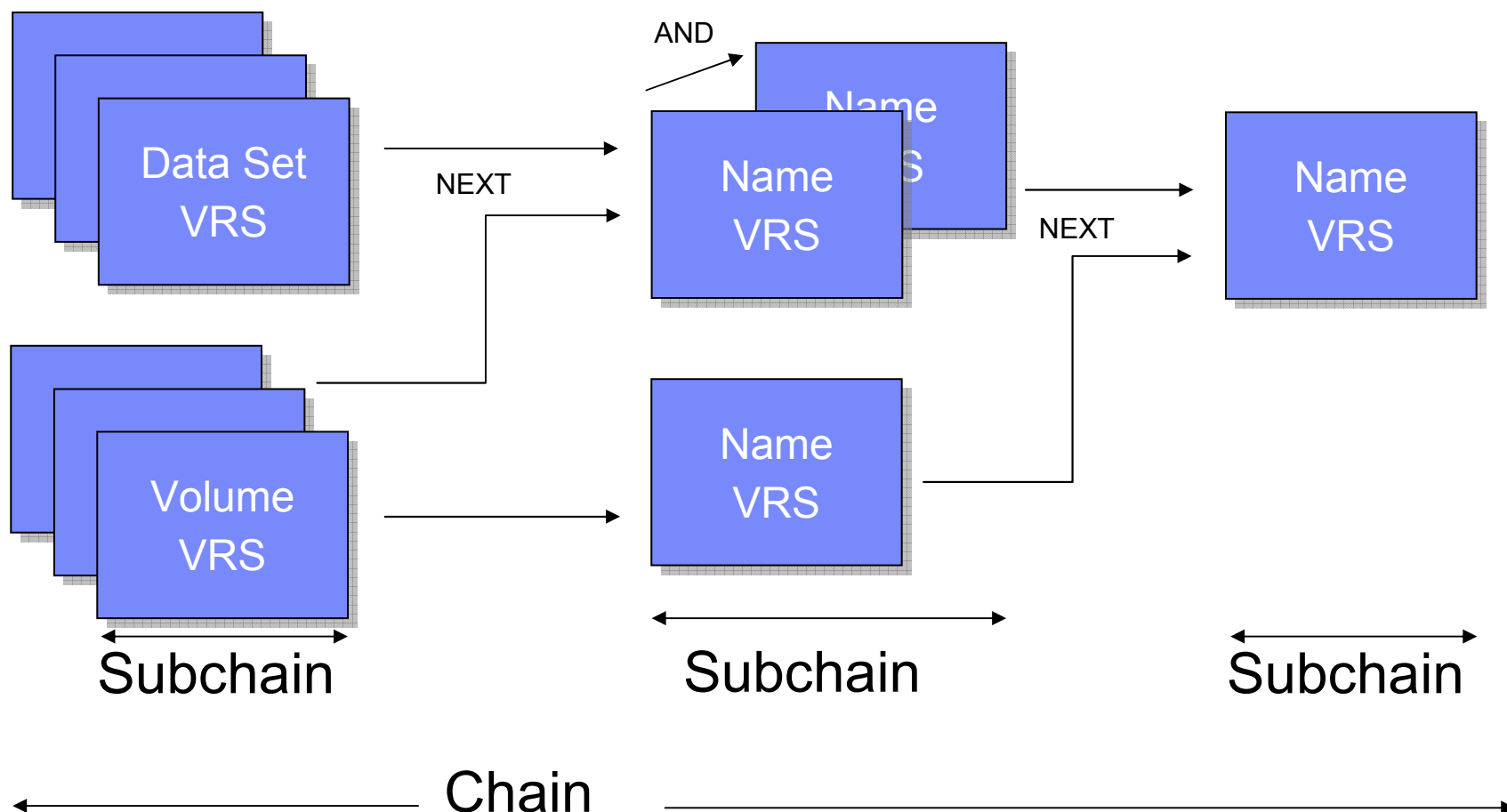


Multiple Retention Types

- **All retention criteria must be met to retain the data set**
 - Retain the data set while it is cataloged ...
 - until 100 days since creation

```
DSN('MC1')  
WHILECATALOG  
  
DAYS  
COUNT(100)
```

Chaining Retention and Movement Policies



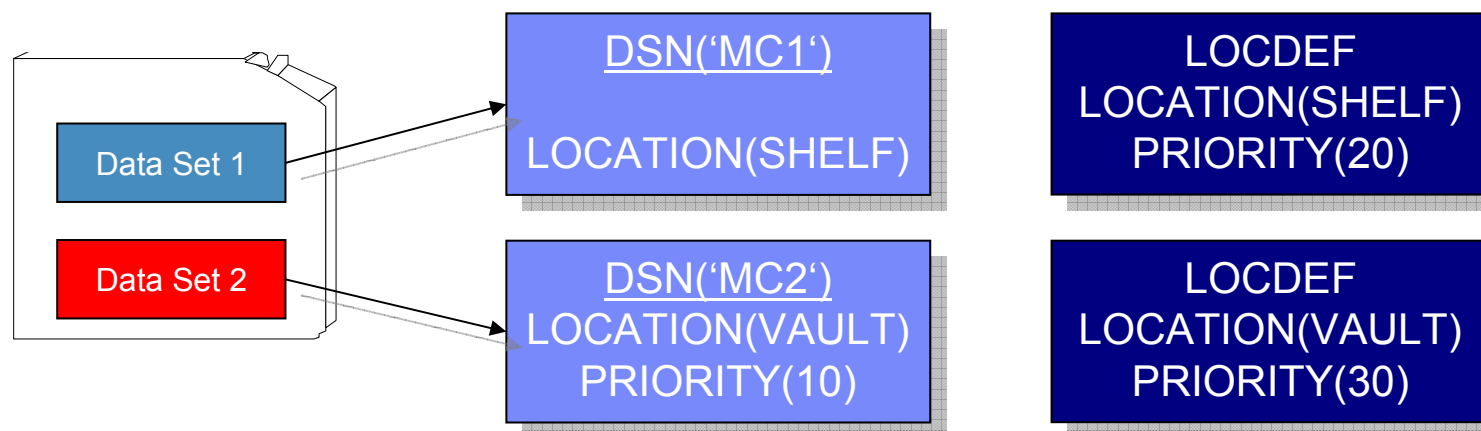
Location

- **Use Location parameter to specify “where” to retain the volume**
 - LOCATION(HOME)
 - Retain in the volume’s home location
 - LOCATION(CURRENT)
 - Retain in the current location
 - LOCATION(location_name)
 - Retain in the library or storage location specified
 - Built in storage locations: LOCAL, DISTANT, REMOTE
 - Installation defined storage locations defined with LOCDEF in the parmlib
- **VRSEL sets the Volume’s required location**
- **DSTORE sets the Volume’s destination and initiates the move**



Location Conflict

- **Solved by Location Priority defined in**
 - EDGRMMxx parmlib member – LOCDEF parameter
 - VRS (overrides LOCDEF)
- **Example: volume retained in VAULT**
 - Lowest number = highest priority



Retention Date

- **Data Set Retention Date**

- PERMANENT
- CYCL/ccccc
- WHILECATLG
- date
- CATRETPD



- **Based upon VRS Subchain which currently retains the data set**

- **Volume Retention Date**

- Highest Retention Date of all Data Set Retention Dates

Release Options

- **Expiry Date Ignore**

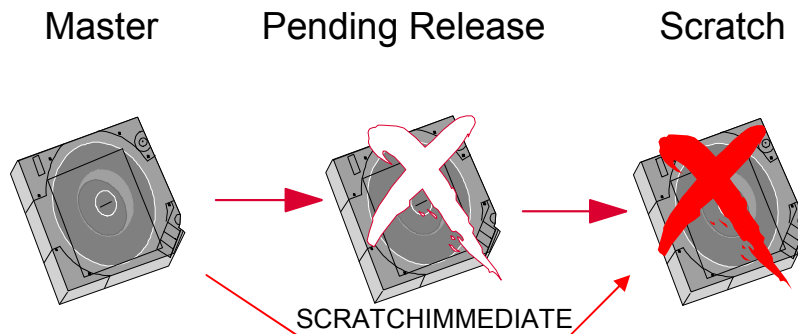
- Do not honor the volume's Expiry Date, once the volume is dropped by VRS

- **Scratch Immediate**

- Return the volume to Scratch immediately during release processing

```
DSN('MC1')  
  
RELEASE(  
EXPIRYDATEIGNORE)
```

```
DSN('MC1')  
  
RELEASE(  
SCRATCHIMMEDIATE)
```



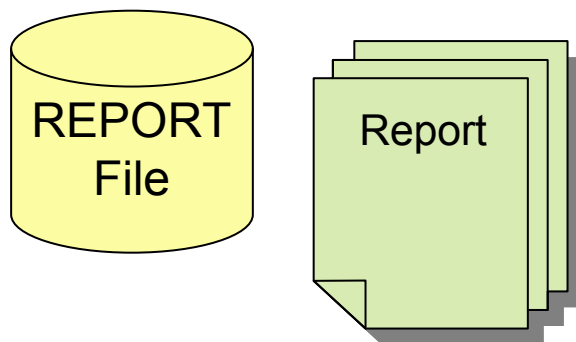
Release options for matching VRSs are applied regardless of whether the data set actually is ever VRS retained. The combination of COUNT(0) and release option changes ensures a Volume can be scratched the same day a Data Set on it is created. (Introduced with z/OS v1.8)

REPORT File

- **Contents**

- Formatted report of all retained data set and volumes
- Sorted by matching Primary VRS

- **Browse it or Print it**



- **Use to identify**

- groups of data sets
- cycles
- what is potentially moved

- **List of Unused VRSs**

- use it to identify and delete VRSs which are no longer required

ACTIVITY File

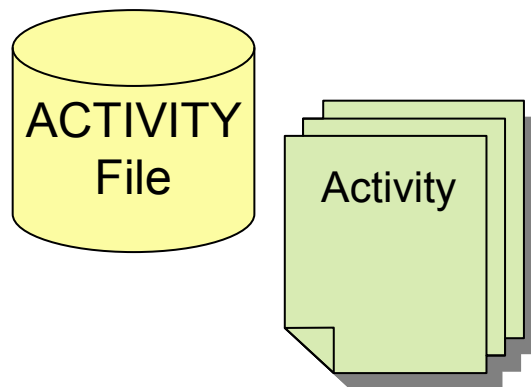
- **Contents**

- Header Record
 - Run Time Values
- Data set Record
 - Details of Changes

- **Browse it or Report on it**

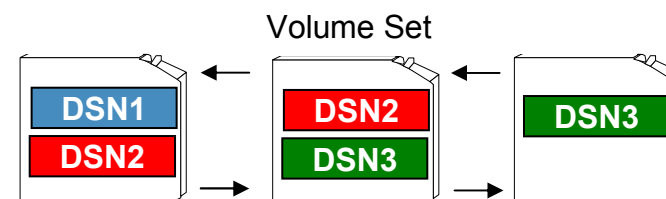
- **Sample Report**

- EDGJACTP
 - Vital status
 - Retention date
 - Matching VRS
 - VRS subchain
- Summaries
- Detailed reports



Recommendation:
Always write ACTIVITY File

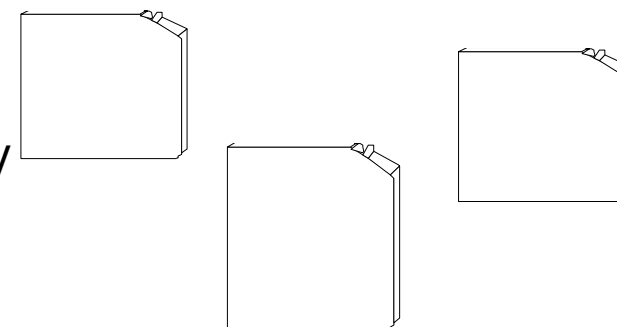
Parmlib Options



■ Volume Set Management

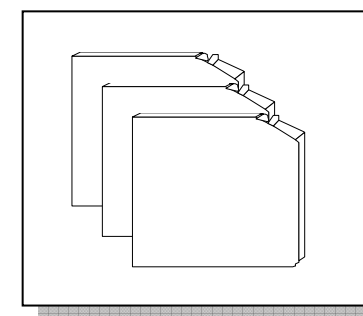
— RETAINBY(VOLUME)

- Volumes of a set are retained individually

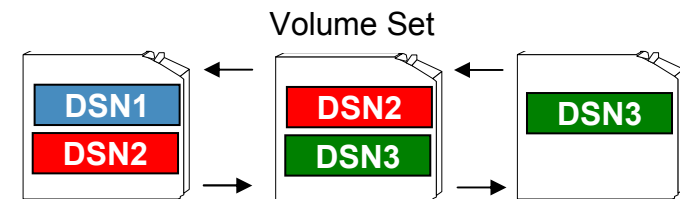


— RETAINBY(SET)

- If any volume in a set is retained by a VRS or Expiration date, all volumes in the set are retained.



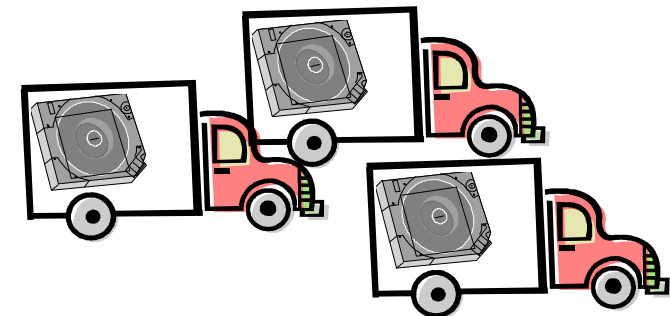
Parmlib Options



■ Volume Set Management

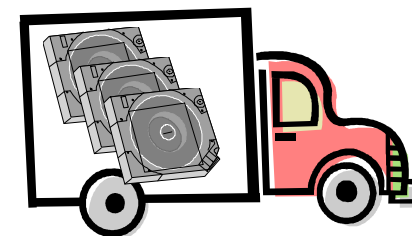
—MOVEBY(VOLUME)

- Volumes of a set are moved individually



—MOVEBY(SET)

- RMM Inventory Management sets the same required location for all volumes of a set which are retained by VRS
- The common location is determined by
 - VRS priority
 - Location priority



Parmlib Options

- **VRSEL(OLD | NEW)**

- NEW: should be used to exploit new functions
- OLD: different matching order, less functions
- The recommendation is to migrate from VRSEL(OLD) to VRSEL(NEW) before moving to z/OS V1R8

Removed in V1R11

- **VRSMIN(number, option)**

- Number: minimum number of policies (default 1)
- Option: Action if minimum not met
 - FAIL
 - WARN
 - INFO
 - OFF

- **VRSCCHANGE(option)**

- Option: first VRSEL run after change of a VRS
 - VERIFY
 - INFO - means: run as production

Parmlib Options

- **CATRETPD(hours)**
 - Hours: catalog retention period (grace period)
- **RETPD(nnnn)**
 - Default retention period for data sets with no RETPD/EXPDT specified in JCL
- **MAXRETPD(NOLIMIT | nnnn)**
 - Controls maximum retention allowed in JCL
- **VRSRETAIN(number, option)**
 - Number: minimum number/percent of newly assigned volumes
 - Option: Action if minimum not met: FAIL, WARN, INFO, OFF
- **VRSDROP(number, option)**
 - Number: minimum number/percent of VRS retained volumes
 - Option: Action if minimum not met: FAIL, WARN, INFO, OFF
- **EXPDTDROP(number, option)**
 - Number: minimum number/percent of EXPDT retained volumes
 - Option: Action if minimum not met: FAIL, WARN, INFO, OFF

Policy Management Trial Run

- **Verify Policy changes**
- **Use to validate VRS adds, changes, deletes**
- **Protects production environment**
- **No CDS updates**
- **Can be used with future date**
- **Forecast retention changes**
- **Activity file required for VERIFY runs**

Recommendation:
Make use of Trial Runs

Example:

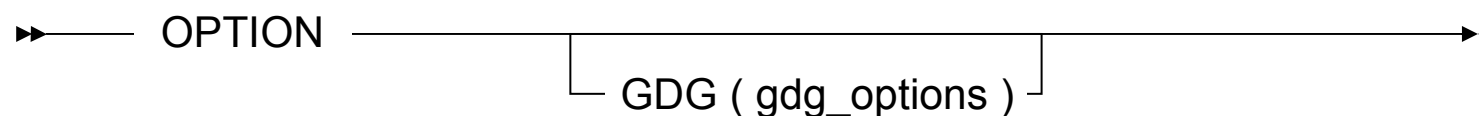
```
//STEP EXEC PGM=EDGHSKP,PARM='VRSEL,VERIFY,DATE=(+10)'
```

Summary of the tasks to perform

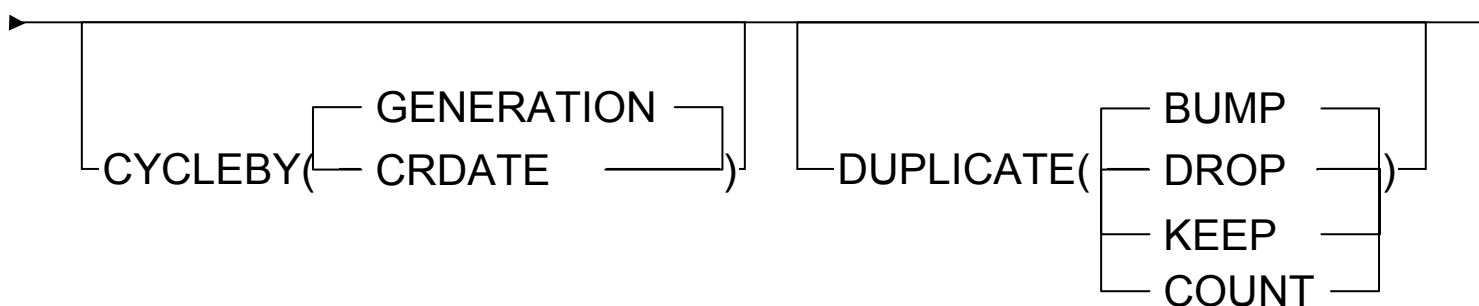
- **Agree Service Levels with Application and User Groups**
 - From these you can determine values for the following:
- **Set installation-wide retention periods in the parmlib member:**
 - RETPD, CATRETPD, MAXRETPD
- **Set other vital record processing options in the parmlib member:**
 - MOVEBY, RETAINBY, VRSJOBNAME, and VRSEL
- **Set Policy Control options in the parmlib member:**
 - VRSCCHANGE, VRSMIN, VRSRETAIN, VRSDROP, and EXPDTDROP
- **Set Media Replacement Rules via MEDINF**
 - Automate Replacement/Migration of Media
- **Define vital record specifications to retain and move data sets and volumes**
- **Define/Create data sets and volumes for DFSMSrmm to manage**
- **Optionally run inventory management in trial run mode**
- **Run DFSMSrmm inventory management vital record processing to process the policies you have defined**

Preview z/OS DFSMSrmm V1R11

- **Policy Management related:**
 - New parmlib options for GDG cycle control and duplicate generation retention



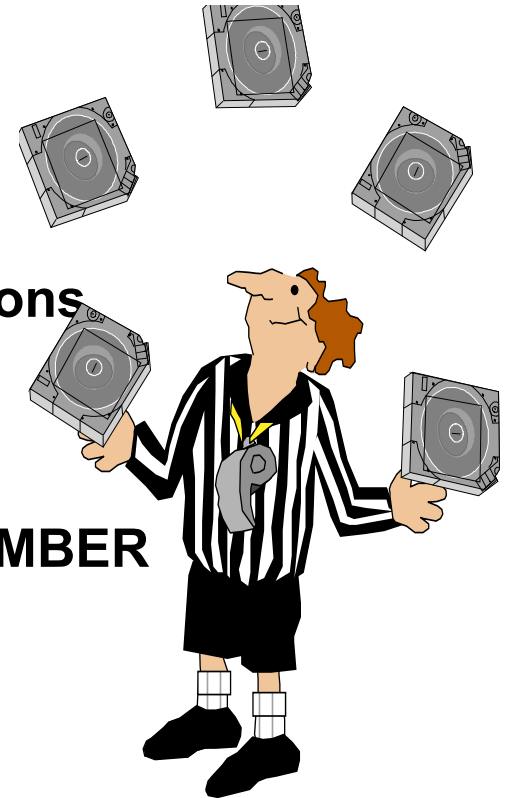
gdg_options



Statements of Future Plans and General Intention. Plans are Subject to Change

Recommendations

- **Exploit SMS Management Class or DSNAME mask VRS's**
- **Establish Service Levels for**
 - Retention and
 - Vaulting
- **Set Retention Expectations and Exception Actions**
- **Reduce number of VRS's**
- **Convert to VRSEL(NEW)**
- **Use Retention Subchains instead of STORENUMBER**
- **Implement CATSYNCH**
- **Set Volume Replacement Rules and Automate Replacement**
- **Consider Enforcement Needs Based on Retention Reasons**



Last but not Least

- **Everything should be made as simple as possible, but not simpler**

